EX18 - Funciones avanzadas - *args, **kwargs y funciones como objetos

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex18_advanced_functions.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Practicar distintas formas de pasar argumentos a una funcion y aprender que una funcion tambien puede guardarse y pasarse como dato.

Variables / elementos del problema

Nombre	Significado
initial_energy	g*h0 + 0.5*v0**2
average_measurements	media de N valores
apply_operation	ejecuta una funcion recibida

Ecuacion o logica en papel

```
energy_0 = g*h0 + 0.5*v0**2
average = sum(values)/len(values)
```

Comprobacion manual

Para h0=100, v0=50, g=9.81: energy_0=981+1250=2231 J/kg. Con g=1.62: 162+1250=1412 J/kg.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- def
- default arguments
- dict
- tuple
- for

Nuevo en este ejercicio:

- positional vs keyword
- *args al definir
- **kwargs al definir
- * al llamar
- ** al llamar
- funcion como objeto
- funcion de orden superior

3. Conceptos Python nuevos

1. *args al definir

Sintaxis: `def average_measurements(*values):`

El * indica que puede llegar un numero variable de argumentos posicionales. Dentro se agrupan en una tuple llamada values.

2. **kwargs al definir

Sintaxis: `def simulation_summary(**kwargs):`

El ** indica que puede llegar un numero variable de argumentos keyword. Dentro se agrupan en un dict.

3. * al llamar

Sintaxis: `average_measurements(*measurements)`

Aqui * hace lo contrario: desempaqueta una secuencia y entrega sus elementos como argumentos posicionales.

4. ** al llamar

Sintaxis: `initial_energy(**config)`

Aqui ** desempaqueta un diccionario en argumentos keyword. Las claves deben coincidir con los parametros explicitos de initial_energy.

5. Funcion como objeto

Sintaxis: `apply_operation(add, 5, 3)`

add va sin parentesis porque queremos pasar la funcion, no ejecutarla en ese punto.

4. Codigo paso a paso

Paso 1. Posicional y keyword

```
initial_energy(h0, v0, g)
initial_energy(h0=1.0, v0=50.0, g=9.81)
```

La primera llamada asigna por orden; la segunda por nombre.

Paso 2. Usar y sobrescribir default

```
def initial_energy(h0, v0, g=9.81): ...
initial_energy(h0, v0)
initial_energy(h0, v0, g=1.62)
```

Si g no se pasa, usa 9.81. Si se pasa, el valor de la llamada sustituye el default para esa llamada.

Paso 3. Recibir *args

```
def average_measurements(*values):
    average = sum(values) / len(values)
```

values es una tuple tanto si llegan 2 como 5 argumentos.

Paso 4. Recibir **kwargs

```
def simulation_summary(**kwargs):
    for key, value in kwargs.items():
        ...
```

kwargs es un dict. items() recorre sus pares clave-valor.

Paso 5. Desempaquetar lista

```
measurements = [10.2, 10.5, 9.9, 10.1]
average_measurements(*measurements)
```

Conceptualmente equivale a average_measurements(10.2,10.5,9.9,10.1).

Paso 6. Desempaquetar dict

```
config = {"h0":0.0, "v0":50.0, "g":9.81}
initial_energy(**config)
```

Importante: initial_energy no define **kwargs. **config convierte las claves en argumentos keyword para parametros normales.

Paso 7. Pasar funcion sin ejecutarla

```
adding_values = apply_operation(add, 5, 3)
multiplying_values = apply_operation(multiply, 5, 3)
```

Dentro de apply_operation se ejecuta operation(x,y). operation apunta a add o multiply segun la llamada.

5. Flujo de ejecucion

```
call styles
 -> packing with *args/**kwargs
 -> unpacking with *sequence/**dict
 -> pass function object
 -> execute it later
```

6. Resultado / comprobacion esperada

apply_operation(add,5,3) devuelve 8; apply_operation(multiply,5,3) devuelve 15. average_measurements acepta distinta cantidad de valores.

7. Errores tipicos al empezar

- Confundir *args al definir con *lista al llamar.
- Confundir **kwargs con **config en una llamada.
- Pensar que initial_energy(**config) significa que initial_energy usa **kwargs.
- Pasar add() cuando queremos pasar add.
- Poner posicionales despues de keyword.

8. Si solo recuerdas cinco cosas

- Posicional se asigna por orden; keyword por nombre.
- *args agrupa posicionales en tuple.
- **kwargs agrupa keywords en dict.
- * y ** en llamadas desempaquetan.
- Una funcion se pasa sin () cuando no queremos ejecutarla todavia.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.